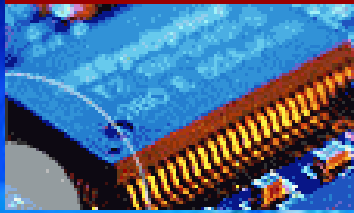


EEE3096S



**Embedded
Systems II**



EEE3096S Exam Study Advice

Embedded Systems II

Study
tips

Dr Simon Winberg



Electrical Engineering
University of Cape Town

Study Advice

Be prepared ...

**It's not just about
finding time to study!**



**Have backups .. and don't
be entirely reliant on
always having connectivity!**

But getting back to Studying...

Particularly for EEE3096/5S exam 2025:

Date: 19 November 2022 Time: 12h30 Duration: 3hrs

Venue: Sports Centre 1

Extra time venue: Hoerikwaggo 3C

Course Notes

The course notes covers the main, more essential parts of the course content, including explanations and examples. Having good familiarity with that will get you to at least 60%. But for getting higher marks, you need more breadth and awareness of embedded systems issues, techniques and design strategies with were discussed in lectures.

The slides that follow indicate suggests approaches for aspects of focus on and be aware of in the Course Notes, followed by aspects of Term 1 lectures that you would want to have awareness of and put some time in to ensuring you have some understanding of.

EEE3095/6S
Embedded Systems II

Official Course Notes



Course Notes

study focus tips

*(aiming to 'split screen' with Course Notes on
One screen, and marked contents on other)*

1 Introduction and Background	1
1.1 What is an Embedded System?	1
1.2 Real-time Systems	3
1.2.1 Characteristics	3
1.2.2 Classification	4
1.3 Development, Execution, and Runtime Environments	6
1.4 Benchmarking	7
1.4.1 Golden Measure	7
1.4.2 Wall Clock Time	8
1.4.3 What to Benchmark?	10
1.5 Optimisation	11
1.5.1 Compiler Optimisation	11
1.5.2 Compiler Flags	12
1.5.3 Bit-widths	13
1.5.4 Optimisation Strategies	13
1.6 Debugging	15
1.7 Embedded Systems Design	16
1.7.1 Design Metrics	16
1.7.2 Design Diagrams	17

Colour coding key: grey = can largely skip violet = know a bit about
 black = know most, consider reading over thoroughly

2 System Architecture 26

2.1 Microcontrollers 26

2.1.1 History (may help; but skip most).. 26

2.1.2 What is a Microcontroller? 27

2.1.3 RISC vs CISC 29

2.2 The Development Board (mainly skip). 30

2.3 Memory Model (understand, but mainly skip) 33

2.3.1 Memory Addresses (understand, need no STM specs) 33

2.3.2 Data Types and Endianness (typically asked) 36

2.4 The ARM Cortex-M0 (largely skip, but have understanding) . . . 37

2.4.1 Programmer's Model of the CPU 37

2.4.2 Basics of CPU Architecture 38

2.4.3 Program Counter 39

2.4.4 Three-stage Pipeline 40

2.4.5 Reset Vector 40

2.5 Computer Architecture (should know this well!) 41

2.5.1 Datapath 41

2.5.2 Function Units 43

2.5.3 Arithmetic Logic Units 46

2.5.4 Control Units 46

2.5.5 Processor Instructions 48

2.5.6 Adaptable Hardware (know what FPGA is, but little asked) 49

3 Peripherals, Exceptions and Interrupts	53
3.1 Peripherals (know general aspects, not STM specific)	53
3.2 Clock Distribution	56
3.2.1 Reset and Clock Control	57
3.2.2 Clock Source	57
3.2.3 Instruction Cycles	57
3.3 General Purpose Input/Outputs (know concept)	58
3.3.1 GPIO Pin Modes (know of, but STM specs not needed)	60
3.3.2 Pull Resistors	62
3.4 Timers	63
3.4.1 Loop Delays	64
3.4.2 Timers Explained	65
3.4.3 General Purpose Timers (know generally, not STM specs)	67
3.5 Exceptions and Interrupts (know concepts, not much STM specifics)	73
3.5.1 Exceptions	74
3.5.2 Interrupts (don't need to know commands, etc for STM interrupts)	76

4 Signals and Systems	81
4.1 Information Theory	81
4.1.1 Hardware in the Loop (won't have much depth)	81
4.1.2 Signals and Sensing	82
4.1.3 Sampling	83
4.1.4 Shannon's Theorem	85
4.1.5 Nyquist's Theorem	86
4.2 Digital-to-Analogue Converters (often asked, sometimes design)	88
4.2.1 DAC Conceptual Design	88
4.2.2 PWM-based DAC	89
4.2.3 R2R Ladder DAC	92
4.3 Analogue-to-Digital Converters	94
4.3.1 Transfer Function	94
4.3.2 Types of ADCs	97
4.3.3 Errors and Calibration (know about)	100
4.3.4 Using the On-board ADC (no need to remember STM use)	104

Note: CH5 (Embedded Communications) comprises many aspects, most of which are very relevant and could be assessed in the exam. But specific use of peripherals and code solutions for STM will not be expected.

5 Embedded Communications 109

5.1 Terminology (know all these well, appears often). 109

(no need to remember circuit or very specific timing requirements for this)

5.1.1 Parallel vs Serial 110

5.1.2 Big Endian vs Little Endian 111

5.1.3 Asynchronous vs Synchronous 111

5.1.4 Simplex vs Half Duplex vs Full Duplex 112

5.1.5 Message Structure 113

5.1.6 Parity Bit 114

5.2 I2C (should know, but no need to remember circuits) 115

5.2.1 Overview 115

5.2.2 Hardware 116

5.2.3 Message Structure 118

5.2.4 Timing (know about request, response, etc., not specific speeds) 121

5.2.5 I2C on the STM32F0 (these specifics not expected) 123

5 Embedded Communications [continued]

5.3 Serial Peripheral Interface (know well)	124
5.3.1 Overview	124
5.3.2 Hardware	125
5.3.3 Message Structure	126
5.3.4 Timing	127
5.3.5 Using Multiple Slave Devices	129
5.3.6 SPI on the STM32F0	131
5.4 UART and RS-232 (useful to know, might get asked)	133
5.4.1 Overview	133
5.4.2 The RS-232 Standard	134
5.4.3 RS-232 Configuration (no need for STM or pin specifics).	136
5.4.4 Message Structure	138
5.4.5 Timing (should know, or be able to work out)	139
5.4.6 UART on the STM32F0 (not expected).	139

Assembly Programming tips:

Should be able to do a simple assembly program, know calling conventions, how to pass parameters and call a function. Unlikely to involve much programming.

6 Assembly Programming 141

6.1 Introduction	141
6.1.1 What is Assembly?	141
6.1.2 Pseudo-Assembly	142
6.1.3 Real Assembly	144
6.1.4 Why Assembly? (should be able to motivate)	146
6.1.5 Compiling (know about, and compiling != interpreting != assembling) ..	146
6.1.6 Registers	147
6.1.7 Linking (little explained on this, covered in EEE4120F)	147
6.1.8 Executing Code	148
6.1.9 A Few Useful Instructions (should at least know these)	148
6.2 Loading and Storing (should know types)	150
6.2.1 Immediate-offset Loading	151
6.2.2 PC-relative Loading	153
6.2.3 Register-offset Loading	153
6.2.4 Storing	154
6.2.5 Accessing Datatypes Other Than Words	154

...

6 Assembly Programming [continued]

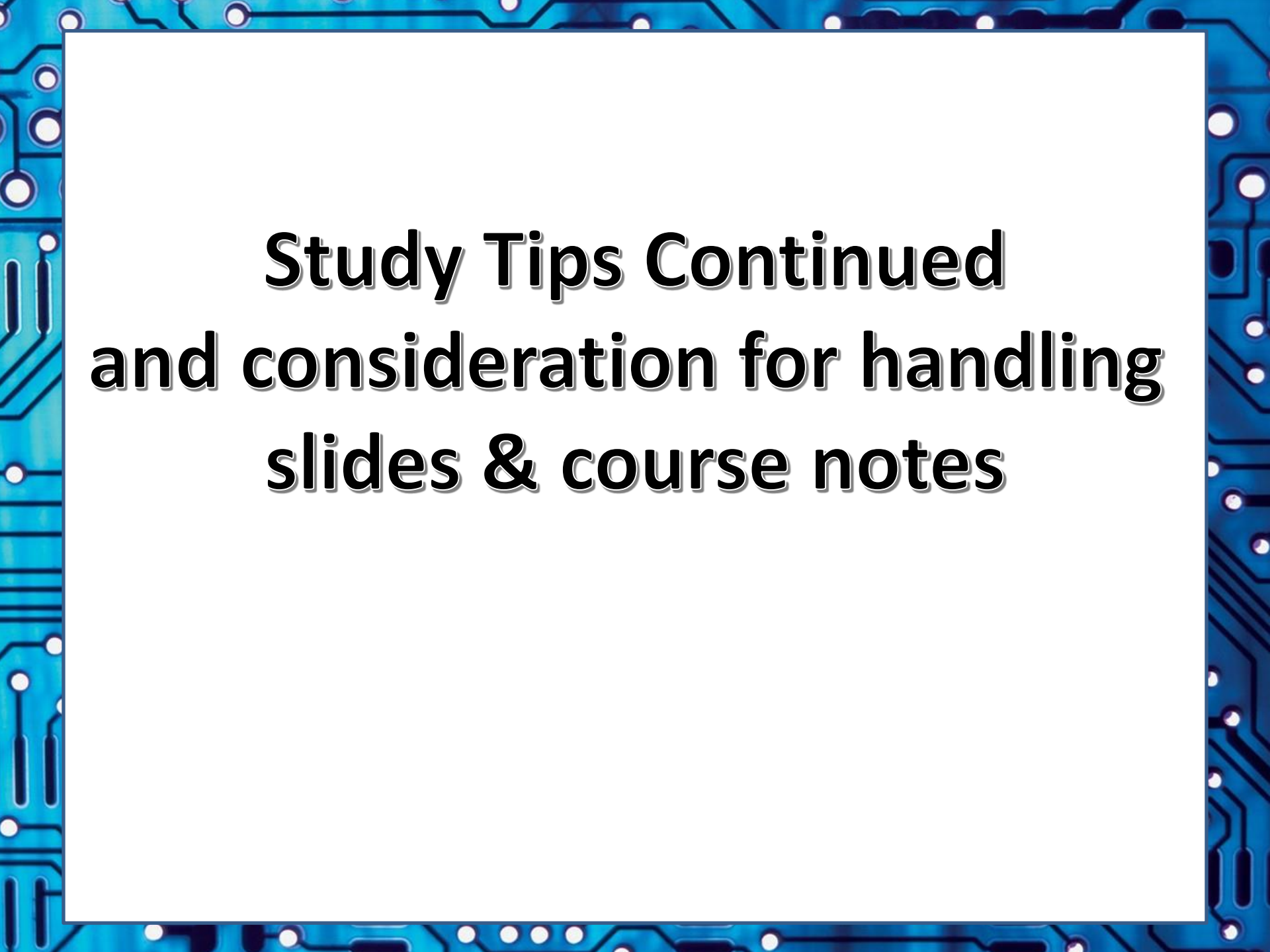
6.3 Branching (should know)	154
6.3.1 Implementation of a Branch	155
6.3.2 Branching via Labels	155
6.4 Conditional Branching	156
6.4.1 Application Program Status Register	156
6.4.2 Overflow Flags	158
6.4.3 The Compare Instruction	160
6.4.4 Condition Code Suffixes	161
6.4.5 Setting or Clearing Individual Bits	161
6.4.6 Branching Based on Individual Bits	163
6.5 Subroutines	164
6.6 Timing	166
6.7 Instruction Sets	167
6.7.1 ARM	167
6.7.2 Thumb (know about, don't need detailed knowledge of this)	168
6.7.3 Thumb-2	169
6.7.4 Implementation of Interworking	169
...	

6 Assembly Programming [continued]

6.8 Stacks	170
6.8.1 Stack Pointer	171
6.8.2 Stack Access Instructions	171
6.9 Mixing C and Assembly	172
6.9.1 Functions in Assembly	172
6.9.2 Calling Conventions	174
6.9.3 Call Stacks	176

Appendices 178

A C Programming 179 : have a look over, mostly aimed to help your understanding of C language, embedded programming methods, and tool use.

The slide features a decorative border with a blue circuit board pattern, including various lines, dots, and geometric shapes. The central content is on a white background.

Study Tips Continued and consideration for handling slides & course notes

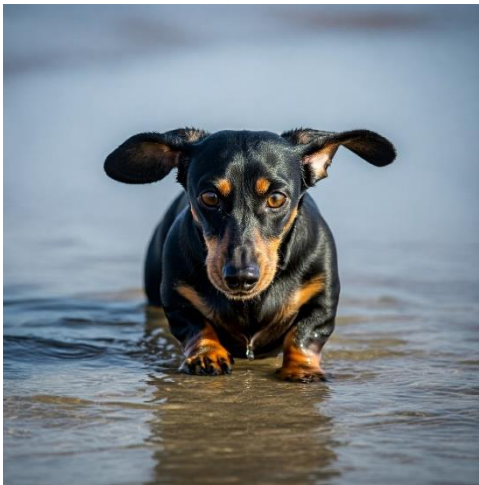
Firstly:

Look over the Course Notes first, i.e. review lecture topics first and then read over the Course Notes sections that relate to the lecture.

Subsequently: (deepening learning method)

After you have read course notes, have more thorough read over the lecture slides (and some of the linked activities / additional reading) in order to gain deeper understand of these topics. Also go over sample solutions and elaborations done in class and past tests.

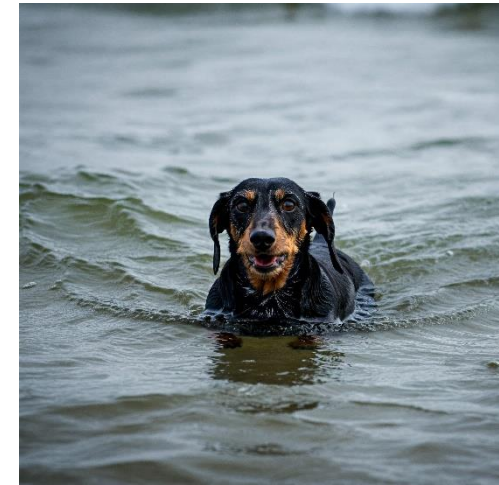
Don't be over strategic, and plan your approach and topics that interests you...



Firstly: Get your toes (or paws) into the topics, know essentials.



Balance your time, but don't be too scared of getting in deeper



More depth can be better answers, but beware of limited study time!

Getting to high marks

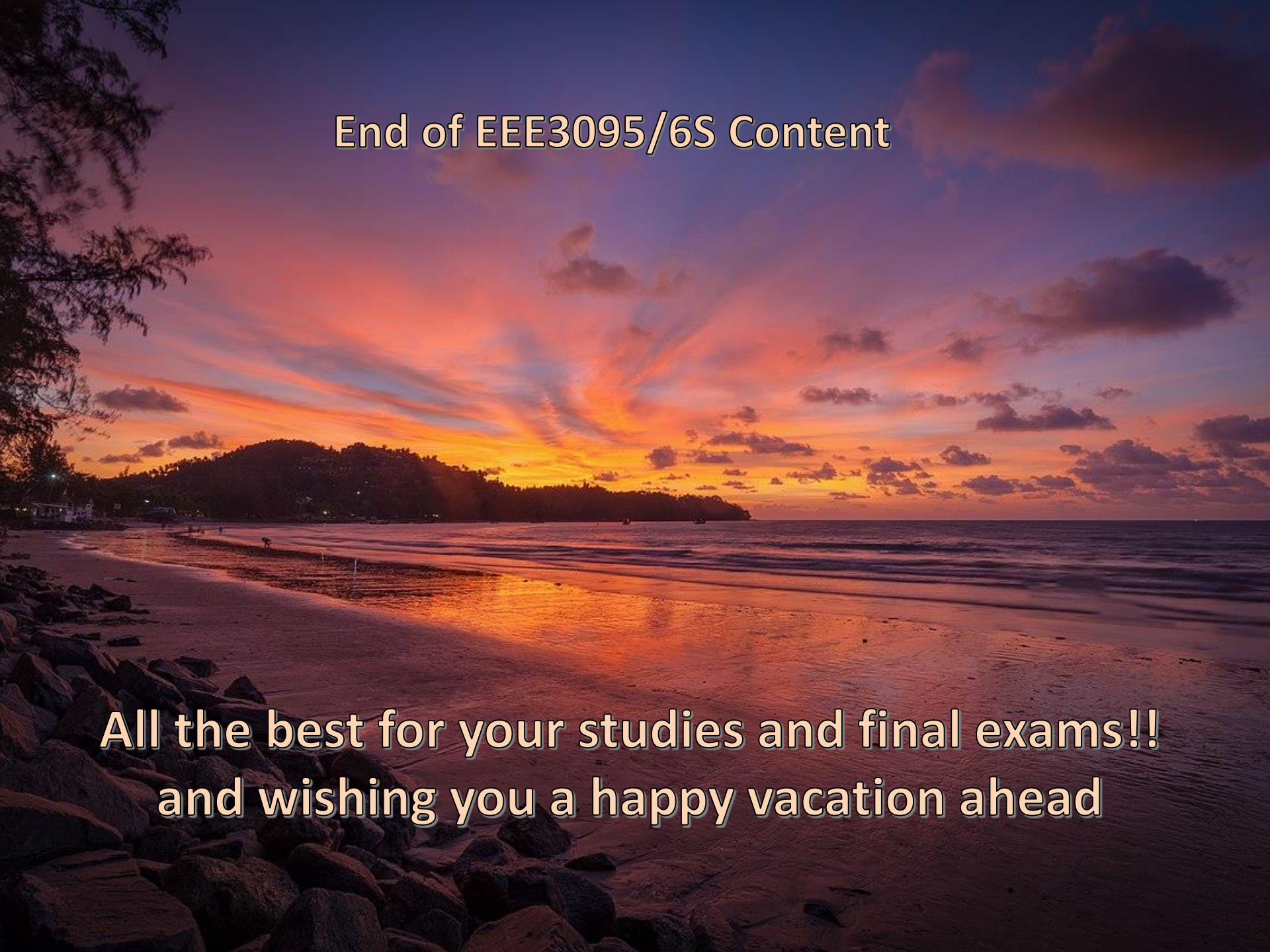


**Towards the marks stratosphere &
some strategic planning**

Getting high marks

- Knowing the theories, terms, tools, etc. is just part of the content, you want to also:
- Work through past tests and exams
 - But try to do the test/exam first and then use the memo to mark your attempt, can help memory and answering strategy
- Know C programming
- Allow some time to add comments in your code
- Good at drawing and understanding design diagrams (i.e., state machines, flow charts, etc.).
 - Knowing and using the right symbols in diagrams can help get top marks for these.
- Write code from diagrams, draw diagrams to explain code or pseudocode
- Do paper-based prototyping (i.e. Section 3 questions)

End of EEE3095/6S Content

A scenic sunset over a beach. The sky is filled with vibrant orange, pink, and purple clouds. The sun is low on the horizon, casting a bright glow. The beach is sandy with gentle waves lapping at the shore. In the foreground, there are dark, jagged rocks. A forested hill is visible in the background under the colorful sky.

End of EEE3095/6S Content

All the best for your studies and final exams!!
and wishing you a happy vacation ahead